

第25回 実践：Web ゲームプログラミング4（敵の出現と移動）

実習時のフォルダ構成

Web_名前 の中に 25 フォルダを作る。第 24 回の sample フォルダと exercise フォルダを、25 の中にコピーして、続きから作る。

```
Web_名前
├── 25
│   ├── sample          ← 第 24 回の sample をコピーする
│   │   ├── index.html
│   │   └── main.js
│   └── exercise        ← 第 24 回の exercise をコピーする
│       ├── index.html
│       └── main.js
```

- sample：講義で使うサンプル（講義内容のコードは、sample/main.js に書き足す）
- exercise：自分のシューティングゲーム（演習 1～2、発展）

講義内容

sample/main.js に書き足す。書いたら sample/index.html をブラウザで開いて確認する。

1. 前回の復習

弾は、次の流れで処理した。今回の敵も、同じ流れで作る。

手順	弾（第 24 回）	敵（今回）
追加する	スペースキーで push	一定の時間ごとに push
動かす	上へ（y -= speed）	下へ（y += speed）
消す	画面の上に出たら splice	画面の下に出たら splice

- 配列から要素を削除する for 文は、後ろから回す（i--）。

2. 今回の main.js の並び

```
// ===== 準備 =====  
// ===== ゲームの状態 =====  
// ===== 自機・弾・敵 ===== ← 敵の配列を追加する  
// ===== キー入力 =====  
// ===== 更新 ===== ← updateEnemies を追加し、update から呼び出す  
// ===== 描画 ===== ← drawEnemies を追加し、draw から呼び出す。drawUI に 1 行追加する  
// ===== ゲームループ =====
```

3. 敵の配列を用意する

「自機・弾・敵」の部分に、`enemies` を追加する。

```
// ===== 自機・弾・敵 =====  
const player = { x: 184, y: 440, w: 32, h: 32, speed: 5 };  
let bullets = [];  
let enemies = [];  
let shotTimer = 0; // 次の弾を撃てるまでのフレーム数
```

4. 敵を出す・動かす・消す (updateEnemies)

「更新」の部分の `updateBullets` の下に、`updateEnemies` を追加する。

```
function updateEnemies() {  
  // 60 フレーム (約 1 秒) ごとに、敵を 1 体出す  
  if (frameCount % 60 === 0) {  
    enemies.push({ x: Math.floor(Math.random() * (canvas.width - 32)), y: -32, w: 32, h: 32, speed: 2 });  
  }  
  for (let i = enemies.length - 1; i >= 0; i--) {  
    enemies[i].y += enemies[i].speed;  
    // 画面の下に出た敵を消す  
    if (enemies[i].y > canvas.height) {  
      enemies.splice(i, 1);  
    }  
  }  
}
```

(1) 一定の時間ごとに出す

- `frameCount % 60 === 0` : フレーム数を 60 で割った余りが 0 のとき、つまり 60 フレームに 1 回 (約 1 秒に 1 回) `true` になる。
- `60` を小さくすると、敵がたくさん出る。大きくすると、少なくなる。

(2) 出現位置を乱数で決める

- 第 18 回で学んだ「`Math.floor(Math.random() * 個数) + 最小値`」の形。ここでは最小値が 0 なので、`+ 0` を省略している。
- 個数を `canvas.width - 32` (キャンバスの幅から、敵の幅を引いた値) にすると、x 座標は 0~367 のどれかになる。敵が画面の右からはみ出さない。

- y 座標を `-32`（敵の高さの分だけ上）にすると、画面の上の外から、すっとうってくるように見える。

(3) 動かす・消す

- すべての敵を、`speed` だけ下へ動かす（y を増やす）。
- 敵の上端（`enemies[i].y`）が、キャンバスの高さより大きくなったら、敵は画面の下に完全に出ていく。`splice` で消す。
- 第 24 回と同じく、削除しながら回すので、後ろから調べる。

5. update と draw に追加する

「更新」の部分の `update` と、「描画」の部分で、次のようにする。`drawEnemies` は `drawBullets` の下に追加する。

```
function update() {  
  frameCount += 1;  
  updatePlayer();  
  updateBullets();  
  updateEnemies();  
}
```

```
function drawEnemies() {  
  ctx.fillStyle = "#ff4444";  
  for (let i = 0; i < enemies.length; i++) {  
    ctx.fillRect(enemies[i].x, enemies[i].y, enemies[i].w, enemies[i].h);  
  }  
}
```

```
function drawUI() {  
  ctx.fillStyle = "#ffffff";  
  ctx.font = "20px sans-serif";  
  ctx.textAlign = "left";  
  ctx.fillText(`FRAME: ${frameCount}`, 10, 30);  
  ctx.fillText(`BULLETS: ${bullets.length}`, 10, 60);  
  ctx.fillText(`ENEMIES: ${enemies.length}`, 10, 90);  
}
```

```
function draw() {  
  drawBackground();  
  drawPlayer();  
  drawBullets();  
  drawEnemies();  
  drawUI();  
}
```

- 「ENEMIES」の数が、敵が出ると増え、画面の下に出ると減ることを確認する。
- まだ当たり判定がないので、弾は敵をすり抜ける。当たり判定は、次回に作る。

演習

`exercise/main.js`（自分のシューティングゲーム）に書く。

演習 1 敵を出す

- 講義と同じように、敵が画面の上から出てきて、下へ動き、画面の下に出たら消えるようにする。
- 敵の色・大きさ・速さ・出現の間隔は、自分で決める。
- 敵の大きさを変えたときは、出現位置の計算（`canvas.width - 32`）と、最初の y 座標（`-32`）の `32` も、同じ値に変える。

演習 2 出現の間隔を確かめる

- 出現の間隔（% `60` の `60`）を 20 や 120 に変えて、敵の数はどう変わるかを確かめる。最後に、自分がよいと思う値にする。

発展（任意）

- 敵ごとに、速さが 1～3 のどれかになるようにする。（ヒント：`push` するオブジェクトの `speed` を、乱数で決める。）